PvP Sentinel Complete

A full-stack Discord moderation and Minecraft PvP community bot starter: dashboard, Discord OAuth login, server settings, AutoMod, cases, reports, appeals, reaction roles, custom commands, welcome messages, Minecraft status, player linking, and raid mode.

Architecture

- `bot/`: discord.js bot, moderation engine, interactive components, DiscordSRV webhook receiver.
- `web/`: Express dashboard with Discord OAuth and Socket.IO live feeds.
- `shared/`: SQLite data access and schema.
- `docker-compose.yml`: single-command local deployment.

Start locally

1. Install Node.js 20+.
2. Copy `.env.example` to `.env` and fill in every required value.
3. Run `npm install` at the project root.
4. Run `npm run deploy` to register Discord slash commands.
5. Run `npm start`, then open `http://localhost:3000`.

Alternatively: `docker compose up --build`.

Discord setup

1. Create a Discord application and bot in the Developer Portal.
2. Enable **Server Members Intent** and **Message Content Intent**.
3. Add `http://localhost:3000/auth/callback` as an OAuth redirect for development; use your public HTTPS domain in production.
4. Invite with `bot` and `applications.commands`; the bot needs View Channels, Send Messages, Manage Messages, Moderate Members, Kick Members, Ban Members, Manage Roles, and Manage Channels where you enable those features.
5. Create private `#mod-logs` and `#reports` channels, a staff-only Appeals category, and a Quarantine role. Put their IDs in `.env`.

Minecraft integration

- `/status` uses the Minecraft Java status protocol.
- For bridge messages, configure DiscordSRV or another Minecraft plugin to POST JSON to `/api/minecraft/event` with header `x-bridge-secret` matching `BRIDGE_SECRET`.
- Expected body: `{ "type":"chat", "player":"Steve", "message":"gg", "uuid":"optional" }`.
- Player linking uses `/link <minecraft_name>` plus a generated code; your Minecraft plugin should verify the code through an authenticated endpoint before marking it verified.

Security notes

- Do not expose `.env`, the bot token, OAuth secret, session secret, or bridge secret.
- Use HTTPS and a persistent PostgreSQL database before public deployment; SQLite is intended for a single process/demo.
- The dashboard checks that the signed-in user has Administrator or Manage Guild permission in the chosen server.
- Keep `#mod-logs`, `#reports`, and appeal channels private.